Software Requirements Specification: SecureVault

Version: 3.0 (Hackathon Master Blueprint)

Hardware Target: NVIDIA RTX 5070 (CUDA enabled)

---

1. Introduction

1.1 Purpose

SecureVault is an intelligent, zero-knowledge secrets manager designed for both standard users and developers. It securely stores credentials while utilizing on-device, GPU-accelerated AI to proactively defend against breaches and audit user behavior, all without exposing plaintext data to external APIs.

1.2 Scope

This specification outlines the three phases of the 24-hour hackathon build:

- **Phase 1:** Zero-Knowledge Core (React + Web Crypto API + Django).
- **Phase 2:** Active Hash Auditing (Custom Python Heuristic Engine).
- **Phase 3:** Local AI Defense (GPU-accelerated PyTorch and Local LLM Honeypots).

---

2. System Architecture

- **Zero-Knowledge Pipeline:** All AES-256-GCM encryption occurs entirely in the React client's browser memory. Django acts as "dumb" storage, receiving only unrecognizable ciphertext.
- **Local AI Edge:** To maintain strict data privacy, AI models (PyTorch RNN and Llama/Mistral) run entirely on the local localhost server, utilizing the host machine's RTX 5070 CUDA cores.

---

3. Folder Structure

(Subject to change during the 24-hour development sprint)

```Plaintext
```

```
securevault_root/
├── README.md
├── frontend/            # React UI & Client-Side Cryptography
│   ├── src/
│   │   ├── components/      # React UI (Dashboard, Auth, Audit Badges)
│   │   ├── utils/
│   │   │   └── vaultCrypto.js # Web Crypto API logic (PBKDF2, AES-GCM)
│   │   ├── App.js
│   │   └── index.js
│   └── package.json
└── backend/             # Django Backend & AI Engines
    ├── manage.py
    ├── securevault_api/     # Django core settings & routing
    ├── vault/           # Main App (Models, Views)
    │   ├── models.py        # VaultEntry (Ciphertext + Risk Metadata)
    │   ├── views.py         # Core endpoints (POST /store, GET /retrieve)
    │   └── urls.py
    └── ai_engine/           # Threat Detection & Defense Modules
        ├── auditor.py       # Heuristic hash identification logic
        ├── honeypot_llm.py    # Local Ollama LLM integration script
        └── pytorch_model.py   # RNN sequence predictor for password strength
```

---

4. Implementation Plan & Functionalities

Phase 1: Core Viability (The MVP)

Objective: Build a functional, mathematically secure vault.

- **Functionality 1.1 (Key Derivation):** The React client derives a 256-bit AES key from the user's master password using PBKDF2 (100,000 iterations).
- **Functionality 1.2 (Local Encryption):** React encrypts the vault payload (JSON) using AES-GCM, outputting Ciphertext, an IV, and a Salt.
- **Functionality 1.3 (Dumb Storage):** Django receives the payload via a POST /store API and saves it to the database linked to the user's JWT session.
- **Implementation:** React Web Crypto API (window.crypto.subtle) talking to Django REST Framework via Axios.

Phase 2: The DevSecOps Auditor

Objective: Actively audit pasted secrets before they are encrypted.

- **Functionality 2.1 (Ephemeral Analysis):** When a user pastes a string, React sends it to an ephemeral Django endpoint POST /audit. Django does *not* save it.
- **Functionality 2.2 (Heuristic Identification):** The auditor.py script checks string length, character sets (hexadecimal), and prefixes to identify developer tokens (JWT, AWS) or legacy hashes (MD5, SHA-1).
- **Functionality 2.3 (Dynamic UI):** The endpoint returns a JSON risk profile. React dynamically updates the UI (e.g., flashing red for MD5 with a warning to rotate the credential).
- **Implementation:** Pure Python string analysis ($O(1)$ complexity) integrated into a DRF API view.

Phase 3: Hardware-Accelerated Defense (RTX 5070)

Objective: Unleash the GPU to create active traps and deep-learning analysis.

- **Functionality 3.1 (Contextual LLM Honeypots):** Upon user registration, Django triggers a local LLM to generate mathematically valid but completely fake enterprise secrets (e.g., fake Stripe API keys). These are silently saved in the database as decoys to trap automated hacking tools.
- **Functionality 3.2 (Deep Learning Predictability Score):** Instead of standard regex password rules, a local PyTorch Neural Network analyzes the user's master password to calculate the statistical probability of a human guessing that exact character sequence.
- **Implementation:** * *Honeypots:* Ollama running a lightweight model (e.g., Llama 3 8B) locally, accessed via Python subprocess or requests in Django.
    - *Predictor:* A pre-trained PyTorch Character-RNN running on CUDA.

---

5. Non-Functional Requirements

- **NFR-1 (Zero-Knowledge Limit):** Master passwords and plaintext vault data must never be transmitted over the network to the backend or external APIs.
- **NFR-2 (Latency):** Cryptographic operations and heuristic audits must complete in < 500ms to prevent UI blocking.
- **NFR-3 (Graceful Degradation):** If the CUDA environment fails or the Local LLM crashes, the system must bypass Phase 3 and fall back to Phase 2 functionality without locking the user out of their vault.